# FineProg + Streaming Mamba-2 历史记忆编码器

网络结构、数据流、维度与实现说明

Design summary for downstream implementation

2026-10-08

**最终选型**。保留 FineProg 的两帧 causal context，单个 raw local embedding $e_t \in \mathbb{R}^{B\times 128}$ 直接成为一个 Mamba-2 token。采用 `d_model=128,d_state=64,d_conv=4,expand=2`，1 层。训练完整消费 Episode 的 $T_b$ 个时刻，再选 $L$ 个位置计算原 SSL loss；在线每个真实观测更新一次 state。首轮只改变 `d_state` 与 `d_conv`，不调 `dt_min/dt_max`。

## 1 设计目标与约束

本设计面向 FineProg 视觉 progress reward model。局部编码器负责当前两帧视觉 context，Mamba-2 积累已观察到的 local embedding 历史；输出仍回到 128 维 FineProg embedding 空间，供原 expert/Gaussian query 与 PBRS 使用。

1. **保留局部视觉主干**。ResNet50 Conv4c、两帧 TemporalEmbedder、Conv3D、pool 与 FC 主体保持原接口。
2. **每个 context 一个 token**。$e_t$ 来自两帧，而 Mamba 接收一个 128 维 embedding；不再拼接多时刻视觉 chunk。
3. **完整 Episode 训练**。从 $t = 0$ 到 $t = T_b - 1$ 连续 forward，采用 full BPTT；不在最大 loss 索引处裁剪历史，不做 TBPTT 或中段随机启动。
4. **监督采样与历史解耦**。$L$ 只控制 loss 目标数，默认沿用 $L = 20$；$T_b$ 控制真实 memory 时间轴。
5. **在线立即更新**。每个新观测推进一次卷积缓存和 SSM state；重复 query 使用已缓存输出。
6. **首轮控制变量**。固定宽度、层数、expand、视觉设置、loss、采样 cadence 与优化配方；仅比较 state 容量和局部卷积宽度。

**时间语义**。Mamba 的学习型 $\Delta$ 是内部离散化参数，不能直接换算成物理上的 40 ms。输入采样间隔由数据与环境定义，见第 5.4 节。

# 2 符号与最终配置

| 符号 / 配置 | 含义 | 固定基线 |
|---|---|---|
| $B$ | batch 中 Episode 数 / 在线并行环境数 | 可变 |
| $T_b$, $T_{\max}$ | 第 $b$ 个完整 Episode 长度 / batch 最大长度 | 可变 |
| $L$ | 参与 SSL loss 的目标位置数 | 20 |
| $c$, $s$ | 两帧 context 的帧数 / 原始观测间隔 | 2, 8 |
| $D$, $M$ | FineProg embedding width / Mamba `d_model` | 128, 128 |
| $a_{\text{exp}}$ | Mamba `expand` | 2 |
| $N$, $k$ | `d_state` / `d_conv` | 64, 4 |
| $d_{\text{inner}}$, $d_h$ | 内部 value width / `headdim` | 256, 64 |
| $n_h$, $G$ | SSM head 数 / `ngroups` | 4, 1 |
| $P_{\text{mem}}$, $E_{\text{mem}}$ | memory token 数 / 宽度 | 1, 128 |
| $K_{\text{scan}}$ | 官方 kernel `chunk_size` | 256 |
| $\alpha$ | 可学习 residual 标量 | 初始 $10^{-3}$ |
| 层数 | 单个官方 Mamba2 block | 1 |

表 1：$c$ 是视觉 context 帧数，$k$ 是 Mamba 内部局部卷积宽度，$L$ 是监督目标数。$K_{\text{scan}}$ 只是计算分块参数，不是视觉窗口或训练截断长度。

```
from mamba_ssm import Mamba2

memory = Mamba2(
    d_model=128, d_state=64, d_conv=4, expand=2,
    headdim=64, ngroups=1, layer_idx=0
)
# One block; keep other official defaults unchanged.
# dt_min / dt_max are deliberately omitted.
```

其他设置沿用固定版本默认值：`d_ssm=None`、`rmsnorm=True`、`norm_before_gate=False`、`use_mem_eff_path=True`。本版固定 `dt_min=0.001`、`dt_max=0.1` 的官方初始化默认值；它们不进入首轮搜索。[R1]

因为 $D = M = 128$，外部 input bridge 默认采用 Identity；Mamba 自己的内部 `in_proj` 仍正常执行。

# 3 第一阶段：FineProg 局部视觉编码

## 3.1 两帧 causal context 与 raw embedding

对 $t = 0, \ldots, T_b - 1$，保留原来的局部 context：

$$V_t = \text{Stack}(o_{\max(t-s,0)}, o_t) \in \mathbb{R}^{B\times 2\times 3\times 224\times 224}, \qquad e_t = \mathcal{E}_{\text{local}}(V_t) \in \mathbb{R}^{B\times 128}. \tag{1}$$

Episode 开头不足 $s$ 个历史观测时，使用第一帧补齐过去帧。完整 Episode 的序列表示为

$$V \in \mathbb{R}^{B\times T\times 2\times 3\times 224\times 224}, \qquad E \in \mathbb{R}^{B\times T\times 128}.$$

$e_t$ 必须是 TemporalEmbedder 的未做最终 normalization 的输出。不能把已 normalization 的旧 encoder 输出当作 raw cache，再在融合后重复归一化。

## 3.2 视觉数据流与维度

| 步骤 | 完整 Episode 路径的 shape |
|---|---|
| 两帧 RGB flatten | $[2BT, 3, 224, 224]$ |
| ResNet50 Conv4c | $[2BT, 1024, 14, 14]$ |
| Regroup / Conv3D 输入 | $[BT, 1024, 2, 14, 14]$ |
| Conv3D $1024 \to 512 \to 512$ | $[BT, 512, 2, 14, 14]$ |
| Global max pool | $[BT, 512]$ |
| FC $512 \to 512 \to 512 \to 128$ | $[BT, 128]$ |
| 恢复 Episode 轴 | $E : [B, T, 128]$ |

固定 backbone 为 eval 时，可对每个原始帧计算一次 $F_t$，再 gather $(F_{\max(t-s,0)}, F_t)$ 构造 context，避免两帧 context 重叠产生重复 backbone forward。该路径需要与直接 context 编码在相同统计、预处理和归一化设置下等价。

首轮固定 local encoder checkpoint 与 preprocessing。默认冻结已训练的局部视觉编码器，缓存完整 Episode 的 raw $E$；不冻结随机初始化的 local encoder。第 9 节说明训练模式与缓存有效性。

# 4 第二阶段：单 embedding token 输入

每个两帧 context 已经聚合为一个 $e_t$。在最终配置 $D = M = 128$ 下，直接令

$$x_t = e_t \in \mathbb{R}^{B\times 128}, \qquad X = E \in \mathbb{R}^{B\times T\times 128}. \tag{2}$$

因此外部 `input_bridge=Identity()`；不再新增 $128 \to 128$ 的输入投影作为首轮变量。Mamba-2 内部的 input projection、causal convolution、SSD scan 与 output projection 保留官方结构。

> 一个 token 来自**两帧视觉 context**，但只含一个 raw embedding。不存在 pending visual chunk、Chunk MLP、partial-prefix 或 peek/commit。在线局部 feature ring 与 Mamba 自己的两类状态缓存仍然需要维护。

# 5 第三阶段：Streaming Mamba-2

## 5.1 训练 forward 与在线 step 的接口

训练采用官方可微序列 forward，从零初始状态处理完整 Episode：

$$Y = \mathrm{Mamba2}(X), \qquad X, Y \in \mathbb{R}^{B\times T\times 128}. \tag{3}$$

训练主路径不传 `inference_params`，不把在线原地更新缓存用于 autograd。同一个完整 Episode 仅运行一次 sequence forward；之后 gather loss 目标。

在线 wrapper 可以接收 $x_t : [B, 128]$，但**官方直接 `step` 的输入与输出都有长度为 1 的时间轴**：

$$x_t^{\mathrm{step}} = \mathrm{Unsqueeze}_1(x_t) \in \mathbb{R}^{B\times 1\times 128}.$$

```
y_step, conv_state, ssm_state = memory.step(
    e_t.unsqueeze(1), conv_state, ssm_state
)
m_t = y_step[:, 0, :]  # [B, 128]
```

每个真实新观测调用一次 step，立即保存状态；同一时刻重复 query 不再推进 state。[R1]

# 5 Streaming Mamba-2（续）

## 5.2 内部 projection 与张量尺寸

固定 `headdim=64`、`ngroups=1`、`d_ssm=None`：

$$\begin{aligned} d_{\text{inner}} &= 2 \cdot 128 = 256, \qquad n_h = 256/64 = 4, \\ d_{\text{conv,channels}} &= d_{\text{inner}} + 2GN = 256 + 128 = 384, \\ d_{\text{inproj}} &= 2d_{\text{inner}} + 2GN + n_h = 512 + 128 + 4 = 644. \end{aligned} \tag{4}$$

| 分支 | 基线 sequence shape |
|---|---|
| 输入 / Mamba output | $[B, T, 128]$ |
| 内部 `in_proj` | $[B, T, 644]$ |
| gate / value | 各 $[B, T, 256]$ |
| 卷积输入 $xBC$ | $[B, T, 384]$ |
| value 按 heads 展开 | $[B, T, 4, 64]$ |
| SSM 的 $B, C$ 分支 | 各 $[B, T, 1, 64]$ |
| raw dt / 学习型 $\Delta$ | $[B, T, 4]$ |

## 5.3 持久 recurrent state 与初始化

$$S_t = (S_t^{\text{conv}}, S_t^{\text{ssm}}), \qquad S_{-1}^{\text{conv}} = 0, \quad S_{-1}^{\text{ssm}} = 0. \tag{5}$$

| 状态 | 基线 shape | 保存内容 |
|---|---|---|
| `conv_state` | $[B, 384, 4]$ | 最近 $k$ 个内部投影 $xBC$ 的卷积缓存 |
| `ssm_state` | $[B, 4, 64, 64]$ | 沿本 Episode 的时间持续更新的 SSM 状态 |

调用官方 `allocate_inference_cache`，保留其 layout 与 dtype。默认卷积缓存随 conv 权重 dtype，SSM 缓存随 input projection 权重 dtype；本版不强制所有缓存为 FP32。卷积开头采用零状态，与训练 causal padding 一致；不要用首 token 重复填满卷积缓存。[R1]

## 5.4 dt 的含义与首轮固定范围

$$\Delta_{t,h} = \text{softplus}(\widehat{\Delta}_{t,h} + b_{\Delta,h}), \qquad A_h = -\exp(A_{\log,h}).$$

`dt_min/dt_max` 用于初始化 dt bias 的范围，不是物理秒数，也不是训练后 $\Delta$ 的硬上下界。首轮不改它们，也不把 $\Delta$ 固定成 $0.04$；dt projection、dt bias 与 $A_{\log}$ 仍按原优化配方正常学习。`dt_limit` 等其他参数也保持默认。40 ms 若是采集间隔，只描述 observation cadence。

# 6 第四阶段：Memory 融合、progress 与 PBRS

## 6.1 Memory projection 与 residual

Mamba-2 的 readout 已是 128 维。保留一个可训练的 memory output adapter：

$$\begin{aligned} Z_t^{\text{mem}} &= \text{Reshape}_{1\times 128}\big(m_t W_{\text{mem}}^{\mathsf{T}} + b_{\text{mem}}\big) \in \mathbb{R}^{B\times 1\times 128}, \\ u_t &= \text{Squeeze}(Z_t^{\text{mem}}) \in \mathbb{R}^{B\times 128}, \\ \widetilde{z}_t &= e_t + \alpha u_t, \qquad z_t = \text{EncoderNorm}(\widetilde{z}_t) \in \mathbb{R}^{B\times 128}. \end{aligned} \tag{6}$$

$W_{\text{mem}} \in \mathbb{R}^{128\times 128}$，$\alpha$ 从 $10^{-3}$ 初始化。最终 normalization 位于融合之后，沿用 checkpoint 的 `none/l2` 契约。

$$\text{EncoderNorm}(v) = \begin{cases} v/\max(\|v\|_2, \varepsilon), & \text{l2}, \\ v, & \text{none}. \end{cases}$$

l2 按最后的 $D$ 维逐向量计算；沿用原 FP32 输出约定。关闭 memory 或令 $\alpha = 0$ 时，恢复原 local-only embedding。

## 6.2 Reference query 与 progress

用新 encoder 完整消费 reference Episode，得到 $Z_{\text{ref}} : [N_{\text{ref}}, 128]$。保留原 expert/Gaussian query 实现；以距离加权 query 为例：

$$d_{t,j} = \|z_t - z_j^{\text{ref}}\|_2^2, \quad w_{t,j} = \frac{\exp(-d_{t,j}/\tau)}{\sum_q \exp(-d_{t,q}/\tau)}, \quad p_t = \sum_j w_{t,j}\rho_j \in \mathbb{R}^B. \tag{7}$$

expert embeddings、PCA、Gaussian 与 calibration 均用新 history-aware 表征重新生成或拟合。Reference 的 $\rho_j$ 与原始长度 metadata 不作为在线 memory 的输入。

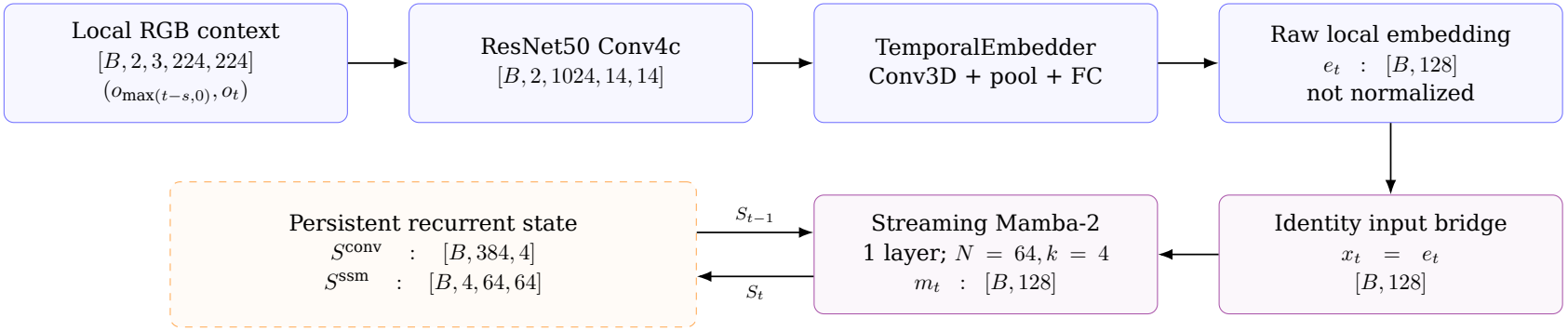
## 6.3 PBRS 与 Episode reset

$$r_t^{\text{train}} = r_t^{\text{base}} + \beta\big[\gamma(1 - d_t^{\text{true}})p_{t+1} - p_t\big]. \tag{8}$$

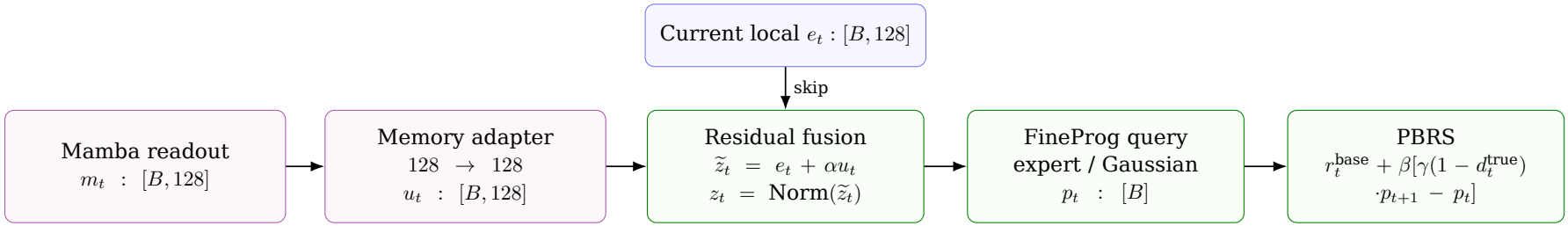
真正终止端 potential 置零；time-limit truncation 按环境 bootstrap 语义单独处理。先使用真实 successor observation 完成 transition 的 potential，再处理自动 reset，避免将新 Episode 初始图像接到上一 Episode。Rollout 期间固定 reward encoder 与 references。

# 7 完整网络结构图与数据维度

完整 Episode 训练与逐观测推理共享参数。一个两帧 context 对应一个 Mamba token，视觉 ring 与 Mamba 内部缓存各自承担固定职责。




(a) 完整 Episode：$[B, T, 128]$ forward；在线：$[B, 1, 128]$ step，每观测一次更新。




(b) 两路共享融合与 progress 接口；训练完成 causal memory 后，才 gather $L$ 个 loss 位置。

图 1：Mamba-2 完整 Episode / 单 token streaming 结构。无视觉 chunk、pending prefix 或 preview state。

| 公开接口 | 训练 / 推理输出 |
|---|---|
| Mamba readout | $[B, T, 128]$ / $[B, 128]$ |
| 最终 FineProg embedding | $[B, L, 128]$（loss）/ $[B, 128]$ |
| 在线 progress / reward | 均为 $[B]$ |

# 8 完整数学数据流

对完整 Episode 的每个时刻 $t = 0, \dots, T_b - 1$：

$$\begin{aligned}
V_t &= \mathrm{Stack}(o_{\max(t-s,0)}, o_t) && \in \mathbb{R}^{B\times 2\times 3\times 224\times 224},\\
e_t &= \mathcal{E}_{\mathrm{local}}(V_t) && \in \mathbb{R}^{B\times 128},\\
x_t &= e_t && \in \mathbb{R}^{B\times 128},\\
(m_t, S_t) &= \mathcal{F}_{\mathrm{M2}}(x_t, S_{t-1}) && m_t \in \mathbb{R}^{B\times 128},\\
u_t &= m_t W_{\mathrm{mem}}^{\mathsf{T}} + b_{\mathrm{mem}} && \in \mathbb{R}^{B\times 128},\\
z_t &= \mathrm{EncoderNorm}(e_t + \alpha u_t) && \in \mathbb{R}^{B\times 128},\\
p_t &= \mathrm{QueryProgress}(z_t; \mathcal{R}_{\mathrm{ref}}) && \in \mathbb{R}^{B}.
\end{aligned} \tag{9}$$

## 8.1 Mamba-2 的 recurrent state 语义

以下针对一个样本与一个 head，省略 batch 轴。$\mathbf{v}_{t,h}$ 是内部 causal Conv 后的 value，$\mathbf{b}_t, \mathbf{c}_t$ 是 $G = 1$ 的 SSM 输入与读出分支：

$$\begin{aligned}
H_{t,h} &= \exp(\Delta_{t,h} A_h) H_{t-1,h} + \Delta_{t,h} \mathbf{v}_{t,h} \mathbf{b}_t^{\mathsf{T}} \in \mathbb{R}^{64\times 64},\\
\mathbf{y}_{t,h} &= H_{t,h} \mathbf{c}_t + d_h^{\mathrm{skip}} \mathbf{v}_{t,h} \in \mathbb{R}^{64}.
\end{aligned} \tag{10}$$

head 结果经过官方 gated RMSNorm 与 output projection 得到 $m_t$。卷积的 $k = 4$ 只控制局部 projected-token 混合；SSM state 持续压缩整个已观察前缀，不会在 $k$ 帧或 kernel chunk 边界自动清零。[R1–R2]

## 8.2 完整 Episode 训练的矩阵表达

$$\begin{aligned}
E &= \mathcal{E}_{\mathrm{local}}(V) \in \mathbb{R}^{B\times T\times 128}, \qquad Y = \mathrm{Mamba2}(E) \in \mathbb{R}^{B\times T\times 128},\\
E^{\mathrm{loss}}_{b,\ell} &= E_{b,I_{b,\ell}}, \qquad Y^{\mathrm{loss}}_{b,\ell} = Y_{b,I_{b,\ell}},\\
Z^{\mathrm{loss}} &= \mathrm{EncoderNorm}\Big(E^{\mathrm{loss}} + \alpha(Y^{\mathrm{loss}} W_{\mathrm{mem}}^{\mathsf{T}} + b_{\mathrm{mem}})\Big) \in \mathbb{R}^{B\times L\times 128},\\
\mathcal{L} &= \mathcal{L}_{\mathrm{SSL}}\big(Z^{\mathrm{loss}}; \text{target_steps} = I,\ \text{seq_len} = (T_b)_{b=1}^{B}\big).
\end{aligned} \tag{11}$$

先完整运行 Mamba，再 gather $E/Y$；下游 adapter、融合与 normalization 按位置独立，可只对 $L$ 个目标执行。训练保留该次完整 forward 的计算图，完成一次 loss backward。

# 9 Training / inference 两条通路与一致性

## 9.1 完整 Episode 与监督采样的 $T \to L$

本版固定 $T_{\text{train},b} = T_b$。$T \to L$ 只选择 loss 输出，不缩短输入 Episode，也不把 $L$ 个稀疏 contexts 当成 memory 的连续时间轴。

沿用当前 `offset_uniform`：从允许的原始时刻均匀随机无放回选 $L$ 个位置，再排序；它不是 `linspace` 等间距采样。默认 offset $a = 1$、$L = 20$。[R4]

$$\begin{aligned} I_b &= \mathrm{Sort}\big(\mathrm{Sample}_{\text{without replacement}}(\{a, \dots, T_b - 1\}, L)\big), \\ I &\in \mathbb{N}^{B \times L}, \qquad T_{\text{train},b} = T_b, \qquad \text{seq_len}_b = T_b. \end{aligned} \tag{12}$$

每次读取重新采样 $I$，在本次 forward 内固定。原始帧索引与完整长度传给 loss；不能用采样序列中的第 $\ell$ 个位置替代 $I_{b,\ell}$，也不能把 `seq_len` 改成 $L$。

**例：**$T_b = 160,\ L = 20$，某次采样的最后一个 target 为 148：

$$\underbrace{e_0, e_1, \dots, e_{159}}_{160 \text{ memory tokens}} \xrightarrow{\text{Mamba2}} Y_{0:159} \xrightarrow{\text{gather } I_b} 20 \text{ loss embeddings.}$$

完整消费 160 个时刻；仍传 `seq_len=160`。即使监督止于 148，也不在 149 处结束 forward。未来尾部不会反向影响更早输出；未连接到监督位置的尾部可能没有该次 loss 梯度，完整 Episode forward 的约定仍然成立。

短 Episode 沿用原 sampler 的末帧重复补齐规则。重复 target 只重复 gather 同一个 $z_t$；每个真实时刻仅消费一次。Loss 补齐不是 memory 观测，也不能延长 $T_b$。

## 9.2 Public batch 与数据缓存

| 字段 / 路径 | shape 与约定 |
|---|---|
| 完整 RGB Episode | `frames_full`: $[B, T_{\max}, 3, 224, 224]$ |
| 完整 raw embedding cache | `raw_embeddings`: $[B, T_{\max}, 128]$ |
| 可选 backbone feature cache | `frame_feats`: $[B, T_{\max}, 1024, 14, 14]$ |
| 真实长度 / 右侧 padding | `episode_lengths`: $[B]$；`history_valid`: $[B, T_{\max}]$ |
| 原 loss metadata | `target_steps`: $[B, L]$；`seq_len`: $[B]$ |
| 身份 | `episode_id`/`video_id`；不同 Episode 状态独立 |

`history_valid` 是 padding 标记，不是 Mamba 的可直接传入 mask。首版按真实长度分组或逐 Episode forward，再 stack 固定 $[L, 128]$ 的 loss 输出。

# 9 两条通路（续）：training

## 9.3 训练 step 的输入、输出与 buffer 管理

| Step | 执行与 buffer / state 生命周期 | shape |
|---|---|---|
| 1 | 读取完整 Episode，采样 $I$；有效历史恰为 $0 \dots T_b - 1$，本次 memory 从零状态开始 | $E : [B, T, 128]$；其他输入见 9.2 |
| 2 | raw cache 直接读取全部 $E$；RGB 路径计算每个 context 的 raw embedding | $[B, T, 128]$ |
| 3 | Identity bridge；完整序列 forward，保留 full BPTT 图，不传 inference cache | $X/Y : [B, T, 128]$ |
| 4 | 在 $E/Y$ 上 gather 同一个 $I$，做 memory adapter、residual 与 normalization | $Z^{\text{loss}} : [B, L, 128]$ |
| 5 | 调用原 Composite/TCC/TemporalTriplet loss；保留绝对索引与完整 $T_b$ | scalar loss |
| 6 | 一次 backward 与 optimizer step；释放图，不跨更新复用旧 state | 更新 memory / adapter 参数 |

训练 buffer 是完整输入 $E$、本次 $Y$ 与 autograd activations；不存在逐 target 的历史副本。官方 full sequence 内部处理卷积与 SSD，无需手写 pending prefix 或在线 mutable cache。

## 9.4 固定首轮 recipe 与完整 Episode 梯度

**首版沿用** `frozen_local_cache`。加载已训练 FineProg checkpoint，冻结整个 local encoder 为 eval，缓存每个完整 Episode 的 raw $E$。每次优化读取全部 $T_b$ 个向量；训练 Mamba-2、memory adapter 与 $\alpha$。所有 $N/k$ 实验使用同一份 raw cache 和同一训练配方。

Cache 记录 checkpoint/hash、raw 输出位置、$D = 128$、$c = 2$、$s = 8$、预处理、观测 cadence、Episode 身份与全部时间索引。它只缓存视觉输入，不缓存跨 minibatch 的 Mamba state。视觉权重、augmentation 或预处理改变后，旧 raw cache 必须失效。

```
# Each forward consumes a complete real Episode.
optimizer.zero_grad()
z_batch = []
for b in range(B):
    e = read_raw_episode(episode_ids[b])  # [T_b, 128]
    y = memory(e.unsqueeze(0))[0]         # [T_b, 128]
    z = encoder_norm(e[I[b]] + alpha * mem_proj(y[I[b]]))
    z_batch.append(z)                     # [L, 128]
loss = ssl_loss(torch.stack(z_batch), original_metadata)
loss.backward()
optimizer.step()
```

**联合训练仅作为后续模式**。`joint_full_episode` 同样完整 forward、无 detach；后续目标可向此前未直接监督的历史视觉位置传梯度。固定 BN running statistics，避免跨时间统计造成未来泄漏。首轮不同时切换 frozen/joint 模式。若没有已训练的 local checkpoint，先完成原 FineProg baseline。

# 9 两条通路（续）：inference

## 9.5 每个新观测一次 advance

| Step | 输入输出 | 状态操作 |
|---|---|---|
| 1 | $o_t : [B, 3, 224, 224] \to F_t : [B, 1024, 14, 14]$ | 写入本环境 feature ring |
| 2 | 两帧特征 $[B, 2, 1024, 14, 14] \to e_t : [B, 128]$ | $t < s$ 时过去帧取首帧 |
| 3 | Identity：$x_t = e_t$；unsqueeze 为 $[B, 1, 128]$ | 不建视觉 chunk |
| 4 | 官方 Mamba2.step 输出 $[B, 1, 128]$，取 $m_t : [B, 128]$ | 两类 cache 各更新一次 |
| 5 | memory adapter $u_t : [B, 128]$；融合 $z_t : [B, 128]$ | 缓存当前 embedding |
| 6 | query $p_t : [B]$；完成相邻 potential 的 reward | 保存 potential 与 valid 标记 |

### Feature ring 与 StreamState

```
StreamState:
    feature_ring: [B, s+1, 1024, 14, 14]  # 9 slots at s=8
    conv_state:   [B, 384, 4]             # Allocated by Mamba2
    ssm_state:    [B, 4, 64, 64]          # Allocated by Mamba2
    observed_steps / episode_id: [B]
    previous_progress / potential_valid: [B]
    cached_embedding: [B, 128]            # Repeated query only
```

$$
\begin{aligned}
q_t = t \bmod (s+1), \quad j_t = \max(t-s, 0) \bmod (s+1), \\
\mathcal{B}_{q_t} \leftarrow F_t, \quad e_t = \mathcal{E}_{\text{temporal}}\big(\text{Stack}(\mathcal{B}_{j_t}, \mathcal{B}_{q_t})\big).
\end{aligned}
\tag{13}
$$

先写入当前 feature，再读取 past/current；不足历史时首帧仍保留在 ring 中。**视觉首帧复制与 Mamba 卷积零初始化是两种不同规则**。 State 由调用者按环境与 Episode 独立持有；新真实观测推进一次，重复 query 不推进。

```
F_t = backbone(obs); ring.write(F_t, t)
e_t = temporal_embedder(ring.get_context(t, s))
y1, state.conv_state, state.ssm_state = memory.step(
    e_t.unsqueeze(1), state.conv_state, state.ssm_state
)
z_t = encoder_norm(e_t + alpha * mem_proj(y1[:, 0]))
p_t = query_progress(z_t, references)
state.observed_steps += 1
```

Reset 仅对指定环境清空 ring、两类 Mamba cache、计数、cached embedding 与 potential valid 标记。先完成真实 successor 的 reward 再自动 reset；等待、遮挡、失败恢复或计算块边界不构成 Episode reset。

# 9 两条通路（续）：完整性、提取与成本

## 9.6 完整 Episode、变长与因果性

**完整性**。一次训练样本必须含同一 Episode 的 $0 \dots T_b - 1$。首轮不做随机中段 crop、`max(I)+1` 前缀裁剪、TBPTT、state detach 或跨 optimizer step 的 state 复用。kernel 的 scan 分块不是梯度截断；如果显存不足，优先减少 Episode batch、梯度累积或使用固定的视觉 cache，不悄悄改成短窗口训练。

**变长**。按真实 $T_b$ 分组或逐 Episode 运行 forward，再 stack $[L, 128]$。右 padding 不是真实输入，不能用于在线 advance；也不能将 padding 后的最终 cache 当作有效 Episode 末状态。未来只有在明确验证官方 packed/seq-index kernel 行为后，才能替换首版分组路径。

**因果性**。视觉 BN running statistics 固定。对完整序列训练时，不以所有未来帧的 batch 统计影响早期 embedding。Mamba 与本地两帧 context 均只使用当前及过去观测；loss 索引、总时长标签不输入 memory。

## 9.7 Offline extraction 与 references

从 Episode 起点完整消费同一 observation cadence 的 raw embeddings，每时刻得到 $z_t$。`sample_all_stride` 与 reference stride 只选择输出位置，不在 encoder 输入之前跳过 memory 观测。离线处理块之间保留有效 feature ring 与 recurrent state；换 Episode 才 reset。eval 下验证完整 sequence 与逐步 step 的 $z_t, p_t$ 等价。

## 9.8 训练计算量边界

| 部分 | 本版代价与比较口径 |
|---|---|
| Memory training | 每个完整 Episode 一次 sequence forward，逻辑 token 数为 $T_b$；没有 $L$ 次历史分支重算 |
| Joint visual training | 每个 Episode 编码 $T_b$ 个 contexts，而原稀疏 loss 路径只编码 $L$ 个；近似比例 $T_b/L$ |
| Frozen raw cache | 视觉完整提取一次；每个优化 step 只训练 memory / adapter 与原 SSL loss |
| Online memory | 每个新观测一次 step；state 大小不随已观察 Episode 长度增长 |

$$\frac{C_{\text{visual,joint}}}{C_{\text{visual,baseline}}} \approx \frac{T_b}{L}, \qquad N_{\text{tokens,train}} = \sum_b T_b, \qquad N_{\text{advance},t} = 1.$$

例 $T_b = 160, L = 20$：joint local context 数约为原路径的 8 倍。相对处理至第 148 帧的 149-token 前缀，完整 160-token Episode 多约 7.4% 的逻辑 token。这些是算量口径，不是实测训练时间；报告 forward/backward、总 step 时间、显存与在线延迟。

# 10 代码集成建议与 public contracts

| 文件 / 区域 | 拟修改内容 |
|---|---|
| `models/temporal_embedder.py` | 主体保留；暴露未 normalization 的 raw projection |
| `models/encoder.py` | raw local 接口；融合后统一 normalization；完整 Episode 序列与 $L$ 检查解耦 |
| `models/streaming_mamba2.py` | 单层 Mamba2、Identity input bridge、memory adapter、$\alpha$；sequence / step wrapper 与 cache reset |
| `dataset_preparation/h5vid_dataset.py` | 提供完整 Episode / 完整 raw cache；保留原 target sampler 与绝对时间索引 |
| `train_encoder.py`, configs | 固定 full Episode / full BPTT；一次完整 forward 后 gather；首轮仅覆盖 $N/k$ |
| `extract_embeddings.py`, reward providers | 完整按时间编码；每环境独立状态；重建 reference / calibration |

### 建议公开接口

```
encode_local_raw(contexts) -> E
    [B,T,2,3,224,224] -> [B,T,128]
forward_episode(E, episode_lengths, target_steps) -> Z_loss
    [B,T_max,128] + [B] + [B,L] -> [B,L,128]
forward_sequence(E, episode_lengths) -> Z_dense
    [B,T_max,128] -> [B,T_max,128]  # Valid positions only
encode_step(obs, stream_state) -> z_t, stream_state
    [B,3,224,224] -> [B,128]
reset_stream_state(stream_state, env_mask)
    env_mask: [B]
```

`forward_episode` 内部按真实长度分组, 不改变输入 Episode 范围。在线 public step 返回 $[B, 128]$, 官方直接 step 的 $[B, 1, 128]$ 由 wrapper 适配。`allocate_inference_cache` 的两个缓存由每环境 state 持有，query 与 advance 明确区分。

Checkpoint 记录 local checkpoint/hash、normalization、$c/s$、$D/M$、$N/k$、expand、层数、headdim、groups、输入 bridge、kernel 配置、未修改的 dt 默认配置、源码 SHA、训练模式、full Episode 标记、cadence 与 reference 版本。修改 $N$ 时必须自动推导内部 projection、conv channels 与 cache shape。

迁移需检查稀疏 $L$-context dataset、旧 cache gather 与 sparse extraction 的全部入口, 确保 memory 始终消费完整连续 Episode；不能只替换模型名而沿用截断输入。

# 11 首轮实验范围与关键验证

## 只改变 state 容量与局部卷积宽度

采用以下单因素对照作为**建议起点**；基线配置由用户最终选型固定。不同时改变宽度、层数、expand、dt 范围或视觉训练方式。

| 实验 | $N$ | $k$ | 变化 / 对照 |
|---|---|---|---|
| Base | 64 | 4 | 固定基线 |
| State-small | 32 | 4 | 只减小 `d_state` |
| State-large | 128 | 4 | 只增大 `d_state` |
| Conv-short | 64 | 2 | 只减小 `d_conv` |
| Conv-mid | 64 | 3 | 只改变 `d_conv` |

`causal-conv1d` 官方快速卷积支持宽度 2、3、4，故首轮建议先在该范围比较，避免更换 kernel 路径混入算时差异。[R3]

$$d_{\text{conv,channels}} = 256 + 2N, \quad d_{\text{inproj}} = 516 + 2N, \quad S^{\text{conv}} : [B, 256 + 2N, k], \quad S^{\text{ssm}} : [B, 4, 64, N].$$

`dt_min`/`dt_max`、`dt_init_floor`、`dt_limit` 均保持默认。dt projection / bias 正常训练；这与调 dt 初始化超参数是不同操作。数据输入 cadence 对所有实验保持一致。

## 实现验收与 profiling

| 测试 | 验收条件 |
|---|---|
| Full Episode | 消费 token 数恰为 $T_b$，与 `max(I)` 无关；无 detach / TBPTT |
| Loss gather / metadata | $E/Y/I$ 索引一致；输出 $[B, L, 128]$；`seq_len` 为完整 $T_b$ |
| Cache / legacy | raw cache 与在线 local raw 等价；$\alpha = 0$ 恢复原 normalization 输出 |
| State / reset | 两类 cache shape、dtype、layout 正确；每观测一次更新；指定环境独立 reset |
| 因果 / 变长 | 修改未来不改变先前输出；padding 不 advance；重复 target 不重复推进 |
| Sequence / step | eval 完整 sequence 与逐步 step 的 $z_t, p_t$ 在记录容差内一致 |
| PBRS | 真终止与 timeout 分开；自动 reset 不污染上一 transition successor |
| 控制变量 | 除 $N/k$ 外配置相同；记录 seed、GPU、dtype、kernel、源码与实际时间/显存 |

验证等待、回退、遮挡、失败恢复与 Episode 初期 progress；比较 local-only 与 memory。容量增大并不保证准确率提升。报告验证指标与端到端训练 / 在线成本。

# 12 给下游实现 Agent 的 Prompt

以下两页可直接交给实现 Agent。最终结构是单层 Mamba-2、单 local embedding token 与完整 Episode 训练。

> 在 V373/FineProg 中实现两帧 local encoder + Streaming Mamba-2。固定 `d_model=128,d_state=64,d_conv=4,expand=2`、1 层；完整 Episode full BPTT。首轮仅改变 `d_state/d_conv`；不调 `dt_min/dt_max`，不解释为物理 40 ms。

开始前阅读 encoder、TemporalEmbedder、backbone、dataset、train、extraction、cache 和实际 reward provider。核对完整 Episode 数据边界、checkpoint normalization 与固定官方源码版本。

1. 保留 `context_size=2`、`context_stride=8`、ResNet50 Conv4c、Conv3D/pool/FC。将未 normalization 的输出定义为 $e_t : [B, 128]$。
2. $D = M = 128$，外部输入 bridge 用 Identity，$x_t = e_t$。不建立多时刻 visual chunk、pending/prefix、Chunk MLP、peek/commit 或逐 target 历史重算。
3. 使用官方 `Mamba2`，固定 `headdim=64`、`ngroups=1`、`layer_idx=0`；其他参数沿用固定版本默认值。内部宽度 256、4 heads、projection width 644、conv channels 384。
4. readout $m_t : [B, 128]$ 经 Linear$(128, 128)$ 得 $u_t$；$z_t = \text{EncoderNorm}(e_t + \alpha u_t)$，$\alpha$ 初始 $10^{-3}$。保持单 memory token $[B, 1, 128]$ 的解释与最终 128 维接口。
5. normalization 统一在 residual 后，保持 `none/l2`、旧 checkpoint 与 FP32 l2 约定。Cache 与正常 local 路径暴露同一个 raw 输出位置。
6. Dataset 提供完整 Episode $0 \dots T_b - 1$，原 `offset_uniform` 只产生 $I : [B, L]$。保留 $L = 20$、绝对帧索引与完整 `seq_len`，不把稀疏 targets 当作 memory 输入。
7. 无论 `max(I)` 是多少，完整消费 $T_b$ 个 tokens。短 Episode 的重复 loss targets 仅重复 gather；padding 与重复 target 都不增加 memory 时间步。
8. 首轮固定 `frozen_local_cache`：加载已训练 local checkpoint，完整缓存 raw $E$；只训练 memory、adapter 与 $\alpha$。所有 $N/k$ 对照使用同一 cache / 配方。无 local checkpoint 时先训练原 FineProg，不冻结随机局部编码器。

# 12 实现 Prompt（续）

9. 完整 Episode 的可微 forward 输入输出为 $[B, T, 128]$；不传 inference cache。完成 $Y = \text{Mamba2}(E)$ 后 gather $E/Y$，再融合为 $[B, L, 128]$，调用原 SSL loss。一次 scalar backward；不做 state detach、TBPTT、中段 crop 或跨参数更新复用 state。
10. 首版按真实 $T_b$ 分组 / 逐 Episode forward，再 stack $[L, 128]$。不要默认任意 `history_valid` 可以直接传给官方 forward；padding 后末 cache 不是有效末状态。
11. 后续若做 `joint_full_episode`，同样完整 forward、保留 memory 到历史视觉的梯度，固定 BN running statistics。视觉参数 / preprocessing / augmentation 改变必须使旧 raw cache 失效。首轮不同时切换视觉训练模式。
12. 在线将 $e_t : [B, 128]$ unsqueeze 为 $[B, 1, 128]$ 调用官方 `step`；返回 $y : [B, 1, 128]$ 和更新后的两类 state，取 $m_t = y[:, 0, :]$。基线 `conv_state` 为 $[B, 384, 4]$，`ssm_state` 为 $[B, 4, 64, 64]$。
13. 使用官方 cache allocator 的 layout / dtype；不强制所有 state 为 FP32。每环境维护 $s + 1 = 9$ 槽 feature ring、两类 memory cache、计数、Episode 身份、potential valid 与 cached embedding。每个新观测 advance 一次；重复 query 不消费输入。
14. 首帧 visual context 用 $\max(t - s, 0)$ clamp；Mamba conv / SSM 初始状态清零。Reset 只清空指定环境；处理块与卷积窗口边界不是 reset。先计算真实 successor potential，再自动 reset。
15. 首轮只改变 $N$ 与 $k$；固定 $M = 128$、expand=2、1 层、heads、groups、cadence 与训练配方。不调 `dt_min/dt_max` 等默认超参数；dt projection/bias 与 $A_{\log}$ 正常学习。不将内部 $\Delta$ 固定为 40 ms 或据 FPS 做数值换算。
16. 离线 extraction 从 Episode 起点按同一连续 cadence 编码；stride 只选输出。使用新 $z$ 重建 expert embeddings，重新拟合 PCA/Gaussian/calibration；rollout 固定 reward encoder。
17. 添加第 11 节测试，并报告 local-only 与各 $N/k$ 配置的验证指标、实际 forward/backward 时间、端到端 step 时间、在线延迟、显存、GPU/dtype/kernel/源码版本与数值容差。不得将逻辑 token 数或 state 元素数变化当作实测加速。

实现前提交：修改文件与职责、全部 public shape contracts、StreamState、完整 Episode 训练路径、在线 step 路径、cache/变长/reset 规则与首轮对照配置。随后在授权范围内完成实现与验证；提交接口说明、配置样例、测试与 profiling 结果及尚未验证项。

# 13 参考实现、版本与理论资料

本版在 2026-10-08 核对官方 Mamba-2 的构造、forward、step 与 cache。以下固定版本用于维度与 API 约定；使用不同依赖版本时重新检查接口和数值一致性。

**[R1] 官方 Mamba-2 模块**

Repository：state-spaces/mamba。固定 commit：`e9594ce1c732d97440f0332fdc43170a2294dbfa`。本版核对 `mamba_ssm/modules/mamba2.py` 的 input projection、depthwise Conv、`step`、`allocate_inference_cache` 与 dt 初始化代码。

固定版本：mamba2.py

官方 README 与 Mamba-2 用法

**[R2] SSD / Mamba-2 论文**

Dao, T. and Gu, A. “Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality”, ICML 2024。本文的视觉串联、residual、full Episode recipe 与实验范围属于项目设计。

https://arxiv.org/abs/2405.21060

**[R3] 官方 causal-conv1d**

Dao-AILab/causal-conv1d。官方 README 标明快速 depthwise causal Conv 支持宽度 2、3、4；首轮建议卷积对照保持在同一受支持 kernel 路径。

https://github.com/Dao-AILab/causal-conv1d

**[R4] FineProg 实际接口与 sampler**

Repository：V373/FineProg。固定 commit：`e9f684427f2f4a219c64b1f4bca43714e7882da6`。沿用两帧 context、raw projection、`offset_uniform`、`clip_len=20` 与原 loss metadata；dataset memory 输入改为完整连续 Episode。

models/encoder.py

models/temporal_embedder.py

dataset_preparation/h5vid_dataset.py

configs_v2/train.yaml

**[R5] PBRS**

Ng, Harada and Russell, “Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping”, ICML 1999。终止 / truncation 规则与实际环境一致。

**版本说明**

沿用原 PDF 的 1–13 节结构、数学公式与版式。最终模型为 Mamba-2、128 维、1 层；保留单 context token 结构，训练完整 Episode。首轮固定 dt 默认配置，仅改变 $N/k$。本文是可实现的设计与 Prompt，不是已完成实现或性能测量报告。